PROJECT AND TEAM INFORMATION

Project Title

SmartFile–Adaptive File Compression & Decompression Using Huffman Coding

Student / Team Information

Team Name: *Team #*	*BitMasters*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Negi , Rishi –* *2510380076* *2510380076@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bansal , Vardan –* *2510372517* *2510372517@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Rautela , Priyansh -* *2510370501* *2510370501@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Saksham , Pragya -* *2510370412* *2510370412@geu.ac.in* 

PROPOSAL DESCRIPTION

Motivation

Nowadays, most notes, assignments, project files, and programs are stored digitally. As the number of files increases, they require more storage space and become difficult to share through email or messaging platforms.File compression helps overcome this problem by reducing the size of files without losing the original data.
We selected this project because it helped us understand how the Data Structures and Algorithms concepts taught in our course can be applied in a real application. In particular, we used Huffman Coding, a lossless compression technique that is based on a greedy approach. It works by assigning shorter binary codes to frequently occurring characters and longer codes to less frequent ones, which helps reduce the overall file size.
During the development of this project, we implemented binary trees, priority queues, and file handling in C++.We also gained hands-on experience with Object-Oriented Programming, dynamic memory allocation, and algorithm implementation.One of the main benefits of this project was that it allowed us to see how theoretical concepts learned in the classroom are used in practical applications. It also helped us understand the importance of data compression in saving storage space and improving file transfer efficiency.

State of the Art / Current solution

Nowadays, several file compression applications such as WinRAR, ZIP, and 7-Zip are widely used to reduce file size and make file sharing easier. These applications use different compression algorithms to store data efficiently while preserving the original content.
Some commonly used compression techniques include:

- ***Huffman Coding:*** *Assigns shorter binary codes to frequently occurring characters and longer codes to less frequent characters.*
- ***Run-Length Encoding (RLE):*** *Compresses repeated data by storing the value and the number of times it occurs.*
- ***Lempel-Ziv (LZ):*** *Identifies repeated patterns and replaces them with references to reduce duplicate data.*

Although existing tools provide good compression performance, their internal implementation can be difficult for beginners to understand. Therefore, we developed SmartFile, a simple file compression and decompression application using Huffman Coding. The project focuses on applying academic concepts such as binary trees, priority queues, file handling, and Object-Oriented Programming to a practical problem.
We also observed that compression may increase the size of very small files due to additional header information. To handle this situation, SmartFile uses a RAW fallback mode, which stores the original data when compression does not reduce the file size.

Project Goals and Milestones

The main goal of this project is to develop a file compression and decompression system using Huffman Coding while applying the data structures and programming concepts learned during our course. To achieve this, we have set the following milestones:

- *Read input files in binary mode so that different types of files can be processed correctly.*
- *Calculate the frequency of each character or byte present in the file.*
- *Construct a Huffman Tree using a Priority Queue (min-heap).*
- *Generate unique binary codes for each character by traversing the Huffman Tree.*
- *Develop custom BitWriter and BitReader classes to efficiently store and read data at the bit level.*
- *Design a simple custom file format (.sfile) that stores important information such as the original file size and frequency data.*
- *Use the stored header information to rebuild the Huffman Tree during decompression.*
- *Decode the compressed data and restore the original file without any loss of information*
- *apply Object-Oriented Programming concepts by creating separate classes such as HuffmanNode, BitWriter, BitReader, and Compressor.*
- *Display compression results, including the original file size, compressed file size, and percentage of space saved.*

By completing these milestones, we aim to gain practical experience in implementing algorithms, working with file handling, and understanding how data compression techniques are used in real-world applications.

Project Approach

We are developing this project in C++ by applying Object-Oriented Programming (OOP) concepts. The project uses Huffman Coding to compress and decompress files without losing any data.

How the Program Works

1. Reading the File

First, the program opens the input file in binary mode and reads it byte by byte. While reading the file, it counts how many times each byte appears and stores this information in a frequency array. This frequency data is later used to build the Huffman Tree.

2. Creating the Huffman Tree

After calculating the frequencies, all bytes that appear in the file are inserted into a priority queue. The two nodes with the smallest frequencies are selected and combined to form a new parent node. This process is repeated until only one node remains, which becomes the root of the Huffman Tree.

3. Generating Codes

The program then traverses the Huffman Tree to generate binary codes for each byte. Moving to the left side of the tree represents 0, while moving to the right side represents 1. Once all codes are generated, the program checks whether compressing the file will actually reduce its size.

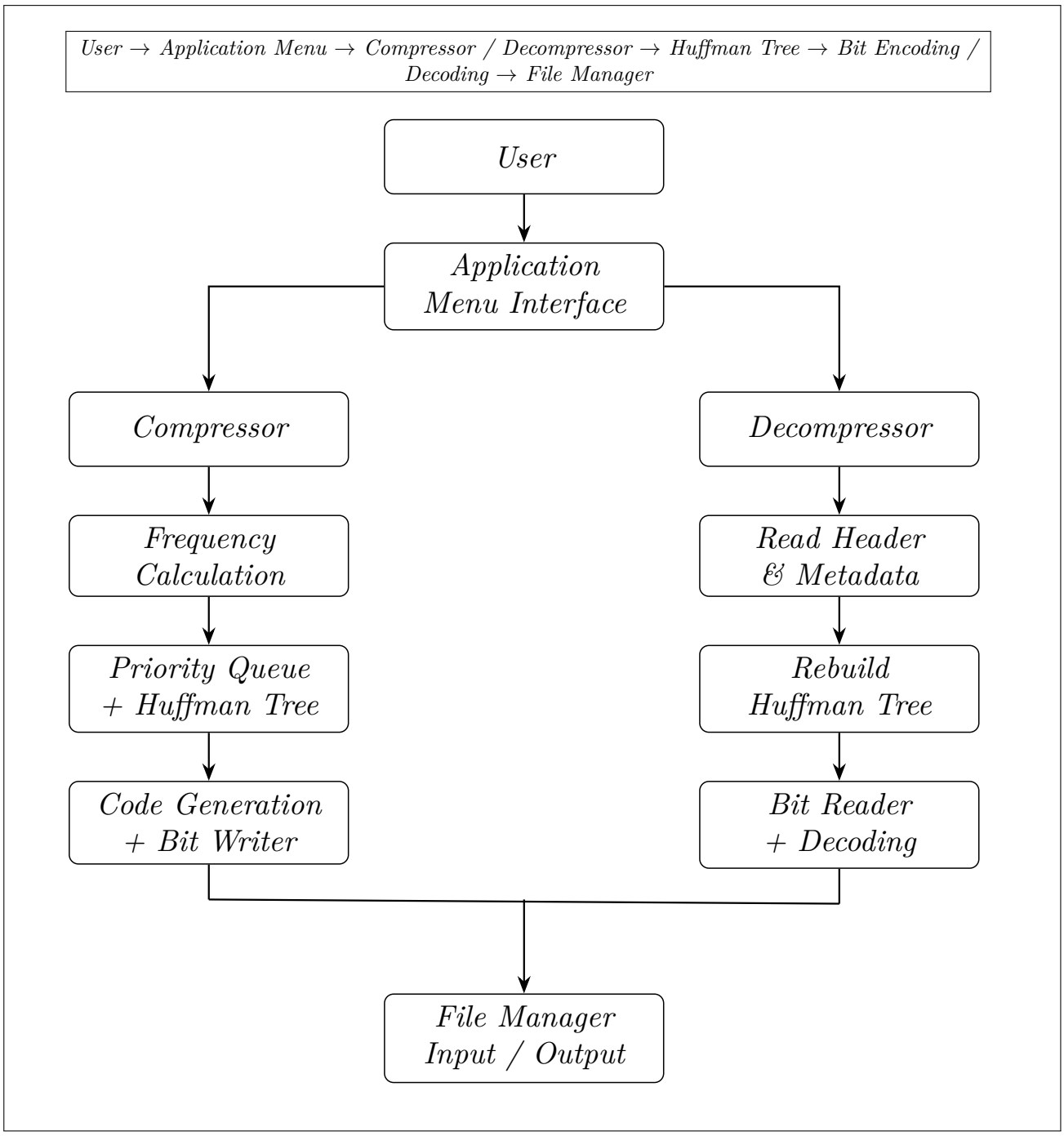
4. Compressing the File

- *If compression results in a smaller file size, the program stores the header information and writes the compressed data using the generated Huffman codes.*
- *If compression does not reduce the file size, the program switches to RAW mode and stores the original file data directly.*

5. Decompressing the File

When decompressing, the program first reads the file header to determine how the file was stored. If the file is in RAW mode, the original data is copied directly. If it is in Huffman mode, the Huffman Tree is rebuilt using the stored frequency information. The compressed bits are then read and decoded to recover the original file exactly.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

At the end of this project, we will have a functional C++ application that can compress and decompress files using Huffman Coding. Through this project, we will also gain practical experience in implementing data structures, algorithms, and file handling concepts. ***Deliverables***

- *Well-organized C++ source code with separate header (.h) and source (.cpp) files for better readability and maintenance.*
- *A working implementation of the Huffman Tree and Priority Queue used for file compression*
- *Custom classes for reading and writing data at the bit level to improve storage efficiency.*
- *A custom .sfile format that stores compressed data along with the required header information*
- *A RAW fallback mode that stores the original file directly when compression does not reduce the file size.*
- *A result screen that displays important information such as the original file size, compressed file size, and percentage of space saved.*
- *Testing and verification results showing that the decompressed file matches the original file exactly, ensuring that no data is lost during the process.*

By completing this project, we will better understand how the concepts learned in Data Structures and Algorithms can be applied to solve real-world problems such as file compression and storage optimization.

Assumptions

The following assumptions have been made while developing this project:

- *The user provides a valid input file, and the system has permission to read and create files.*
- *Huffman Coding is a lossless , so the original data can be recovered completely after decompression .*
- *The compressed file is assumed to be generated by our program and not damaged or modified.*
- *Compression performance depends on the file type. Text and repetitive files may achieve better compression, while already compressed files such as .zip and .png may show little reduction.*
- *Sufficient storage space is available for compressed and decompressed files.*

These conditions define the basic requirements for the system to work correctly and produce accurate results

References

- *Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes.*
- *Weiss, Mark Allen. Data Structures and Algorithms in C++.*
- *C++ Reference Guides , including cppreference.com , for std :: priority_queue and file handling functions.*